

CINAMATE

The Operator's Manual

A PRIVATE STUDIO SYSTEM · 3 CHAPTERS

CONTENTS

01	Getting Started & The Studio Gate	3
02	Projects, The Vault & Cloud Sync	8
08	The Producer Suite I — The Budget Top Sheet	12

Getting Started & The Studio Gate

Cinamate is a private studio system. It has five keys, one door, and twenty-eight rooms behind it. This chapter is about the door: who holds a key, what the lock protects, how long a key stays warm, and what you are looking at once you are through.

About this manual. Every statement here was established by reading the source in this repository, not by operating a running site. The platform is under active development and the code moves faster than the page you are holding. Where this manual and the software disagree, the software is right. Where a behaviour could not be established from source, this manual says so rather than guessing.

Two doors, and only one of them opens

A Cinamate deployment publishes two quite different things. The first is a small public shell: the landing page, `login.html`, the brand assets, and a short allow-list of shared files the landing page itself loads. The second is the studio, and the studio is not published at all. It is bundled *inside* a serverless function.

`scripts/deploy_cinamate.mjs` makes that split at release time: anything not on the public allow-list is moved into the payload of `netlify/functions/gate.js`, and the redirect rules are rewritten so those paths resolve to the function. The consequence is the security model in one sentence — **there is no "view source" route into Cinamate**. An anonymous visitor asking for a module page is not shown a page with the buttons greyed out. Those bytes are not on the public CDN at all. They exist only inside the function, and the function reads the cookie before it reads the file.

The sign-in page

`login.html` is a single card headed CINAMATE, with the line *Production access — owners only* beneath it, fields labelled Login and Password, and a button reading **Enter the studio**. Submitting posts the name and password as JSON to `/.netlify/functions/verify-owner`. Two query parameters change its behaviour: `?u=` pre-fills the Login field and moves the cursor to the password box, accepting only two letters followed by three digits; `?to=` is where you land afterwards, defaulting to `/dashboard.html`. The page resolves that destination and refuses anything that is not a plain same-origin path, so a crafted link cannot use the sign-in card to bounce you off the site.

On success the page keeps only a display record — the owner short and a timestamp — in `sessionStorage` under `cinamate.session`. It never sees the session token, which is deliberately absent from the response body: anything this page's script can read, injected script could read too.

The five owner accounts

Five names are valid and no others. The list appears in `verify-owner.js`, `gate.js` and `projects-sync.js`, and all three check it independently.

LOGIN	PASSWORD LIVES IN
mz465	OWNER_PW_MZ465
kz465	OWNER_PW_KZ465
hz465	OWNER_PW_HZ465
rz465	OWNER_PW_RZ465
dz465	OWNER_PW_DZ465

The passwords are Netlify environment variables on the deployed site. They are not in this repository, not in any page, and not in this manual. A sixth variable, `OWNER_TOKEN_SECRET`, is the HMAC key the system signs and verifies sessions with. Names are matched case-insensitively and both sides of the password are trimmed of edge whitespace, because a pasted secret often carries a stray newline.

When sign-in refuses

Every failure returns the same message — *Invalid name or password* — after the same short random pause, whether the name exists or not, so the response cannot be used to discover which logins are real.

Two throttles sit in front of it. The first counts failures per client address: more than **12 in a 10-minute window** and the endpoint answers 429 with *Too many attempts — wait a few minutes and try again*. That count is kept both in the function instance's memory and in a shared Netlify Blobs store named `cinamate-auth`, keyed by a digest of the address rather than the address itself. The second counts failures per *name* across every address: past **20** in the same window, answers for that name are delayed by 1.5 seconds. It is a delay and never a lockout, because a lockout would let anyone shut an owner out of their own studio by typing a wrong password often enough.

The throttle fails open by design. If the shared counter cannot be reached, the system falls back to the weaker in-memory limit rather than refusing everyone. A storage outage must not become a denial of service against the only five people who can get in.

The twelve-hour session

A correct password mints a token of the form `owner:name:expires:hmac`, valid for **12 hours**. It returns as two cookies, both with a 43,200-second lifetime:

COOKIE	CARRIES	READABLE BY PAGE SCRIPTS
<code>cin_owner</code>	the signed session token	No — HttpOnly
<code>cin_who</code>	the owner short, for UI labels only	Yes

On every request the gate re-derives the HMAC, compares it in constant time, checks the expiry, and checks the name against the list of five. It reads *every* `cin_owner` the request carries, up to twelve, and accepts whichever verifies — so a planted cookie cannot stand in front of your real session and quietly sign you out.

When the twelve hours lapse, nothing dramatic happens; things simply stop being served. A request for a page is redirected to `/login.html?to=` plus the path you were on, so signing in again returns you where you were. A request for a script, stylesheet or image gets a bare 401 `Sign in required`. A call to the studio cloud comes back as *Sign in to use the studio cloud*.

Two limits of the expiry, both read from source. There is no renewal path — `verify-owner.js` mints a token only for a POST carrying the password — so a long day means signing in twice. And a tab left open can look signed in when it is not: the Operator label is drawn from a `sessionStorage` record whose age nothing checks. Nothing in the source warns you before a session lapses, and this manual will not pretend otherwise. If a page starts failing to save, reload it; you will be sent to the sign-in card.

The **Exit** control at the foot of the rail clears the display record, expires both cookies, posts a `logout` so the `HttpOnly` cookie is cleared authoritatively, deletes the service worker's caches so the next person at that browser starts cold, and returns to the public front page. That logout is same-origin only and is deliberately exempt from the throttle: ending a session on a shared machine must never be rate-limited.

The dashboard, and the rail

Signing in lands you on `/dashboard.html`. The status bar across the top carries four readings: a per-session identifier of the form `SES-` plus eight hex characters; **Operator**, showing your owner short; a workspace segment naming your local render bridge if `SB_LocalGPU_v1` holds one, and otherwise reading `WORKSPACE READY`; and a UTC clock.

Below it is the Producer Suite — portfolio tiles, a productions table, a stage badge per title. These read this browser's own saved module stores, refresh every thirty seconds and whenever the tab regains focus, and stay blank until real work exists. The header stamp reads *LIVE FROM THIS BROWSER*, and it means it: what you see here is the machine you are sitting at, not the studio's whole catalogue.

A stale comment to ignore. `dashboard.html` still carries a long header block describing a simulation engine that ticked invented figures upward on timers. The code below it does the opposite and says so. The tiles are real; the comment is a leftover.

Down the left is the rail, and the rail is how you reach everything. It holds twenty-nine entries in six labelled groups, resolving to twenty-eight distinct module pages — *Sales* is a deep link into the budget module rather than a page of its own. Under 900 pixels the rail becomes a bottom tab bar.

GROUP	ENTRIES (PATH)
Develop	Writer <code>/writer/</code> , Boards <code>/boards/</code> , Clear <code>/clearance/</code> , Invest <code>/investors/</code>
Prep	Budget <code>/producer/</code> , Deals <code>/contracts/</code> , Casting <code>/casting/</code> , Scout <code>/locations/</code> , Wardrobe <code>/wardrobe/</code> , Props <code>/props/</code> , Sets <code>/sets/</code> , Safety <code>/safety/</code>
Shoot	Studio <code>/timeline/</code> , Office <code>/production/</code> , Money <code>/finance/</code> , Tools <code>/tools/</code> , Dailies <code>/dailies/</code> , Today <code>/today/</code>
Post	Editor <code>/editor/</code> , Screen <code>/screening/</code> , VFX <code>/vfx/</code> , Music <code>/music/</code> , Post <code>/post/</code>
Release	Sales <code>/producer/#sales</code> , Deliver <code>/distribution/</code> , Fests <code>/festivals/</code> , Tax <code>/taxcredit/</code>
Always	Workflow <code>/workflow/</code> , Projects <code>/projects/</code>

One studio, five owners

A *production* in Cinamate is a named slot holding a snapshot of every module store — screenplay, boards, budget, schedule, cast, cut — together with a manifest of its photographs. The vault engine at `projects/lib-vault.js` creates them, switches between them, and packs them as portable `.cinamate` archive files.

The Projects module also pushes a production to the **studio cloud**, and this is the fact that most changes how the platform feels: the cloud is shared, not personal. There is one catalogue for the site and all five owners see all of it. A pushed production is listed to everyone alongside the owner who last saved it; anyone signed in can open it, and anyone signed in can remove it from the shared list — earlier copies are retained, and the confirmation prompt says so. Pushing under an existing name replaces the cloud copy. The transport accepts up to 4 MB of written record per production and up to 48 MB of media in at most 48 parts, with a 900 KB ceiling on any single file: it carries stills and short takes, not source masters. Anything over those limits is refused whole, with the offending file named, rather than uploaded down to whatever happened to fit.

What stays on this device

Everything the platform saves lives in this browser first, under keys of the form `SB_<Module>_v1` in `localStorage` — sixty-odd across the twenty-eight modules — plus binary media in IndexedDB for wardrobe continuity photographs, scout photographs and cutting-room sources. The project index itself is `CIN_Projects_v1`.

Two keys are deliberately excluded from every snapshot, archive file and cloud push: `SB_LocalGPU_v1`, holding your render bridge address and its API key, and `SB_TMDB_v1`, holding a personal API key. They belong to the machine, not to the film, and nothing carrying a credential travels to another owner.

Clearing this browser's site data discards work. Any production not pushed to the studio cloud or exported to a `.cinamate` file exists nowhere else. Note also that a production opened on a second machine is a copy — it does not stay in step with the first until somebody pushes again.

Where to begin

Sign in, and take the rail from the top. **Writer** is where a production acquires its first real content, and the dashboard tiles stay blank until it does. **Projects**, at the foot of the rail, is where a production is named, snapshotted, archived and shared. The chapters that follow take the modules in that order.

Projects, The Vault & Cloud Sync

A production is not a document you open. It is the whole of this browser's storage, held under one name — and the Projects page is the only place that can move it, copy it, or take it away.

On this chapter. Everything here was established by reading the platform's source rather than by operating it. The software is under active development; if a screen contradicts a paragraph below, the screen is what is running and this page is out of date.

What a production is made of

Two stores hold a film, and they behave differently.

The written record lives in this browser's localStorage, under keys matching `SB_<Module>_v<n>` — the pattern `KEY_RE` in `projects/lib-vault.js`. Script and scenes (`SB_Timeline_v1`), boards, budget (`SB_BudgetSheet_v1`), crew (`SB_Crew_v1`), schedule, clearances, the cut, and some seventy others. A key that does not match that pattern is not part of any production and the vault will not carry it.

The photographs live in IndexedDB, in three databases the vault names explicitly in `BLOB_DBS`: `cinamate_wardrobe` (continuity stills), `cinamate_scout` (location scouting), and `cinamate_cut` (cutting-room sources). Those bytes are never copied into localStorage. A production claims a photograph either because the record carries its project stamp, or because one of its `SB_*` stores still points at that id — which is how a scout photo, which carries no stamp at all, stays attributable to the right film.

The index of every production on the machine is `CIN_Projects_v1`: the active name, and a slot per project holding a full copy of that project's `SB_*` values plus a *manifest* of its media — ids and sizes, no bytes. Bytes stay in IndexedDB deliberately; five photographs written into the one record that indexes every project would exhaust the storage quota and take all of them down together.

Switching between productions

Open takes the outgoing project's live workspace, writes it into that project's slot, and only then rewrites localStorage from the incoming slot. The order matters: the snapshot is persisted before anything is cleared, so an interruption cannot leave a slot pointing at work that has already been thrown away. IndexedDB is not cleared on a switch — the other production's photographs stay on the machine, attributed to it by its manifest.

A name may be reused for nothing: starting a new project under an existing name is refused, and so is renaming onto one. Every module carries the active-project badge from `js/project-badge.js`. If another tab switches productions, open pages lock behind a reload prompt so two films cannot bleed into one another.

Backing up: the .cinamate export

Export backup writes a single file — `format: "cinamate/1"` — containing the project name, a timestamp, every portable `SB_*` value, the media bytes as data URLs, and `blobsMissing`: a list of files the production references that this machine could not produce, recorded at pack time so a later restore can tell "this backup never had them" from "this restore lost them". A cutting-room source above 16 MB is counted and reported, never read into the file.

Four things are deliberately absent.

NOT IN A BACKUP	WHY
<code>SB_LocalGPU_v1</code>	This workstation's render-bridge address and its API key. Machine-local, and a credential.
<code>SB_TMDB_v1</code>	A personal API key. Same reason.
<code>CIN_Learn_v1</code>	Cross-project learning. Outside the <code>SB_</code> prefix, so structurally excluded — it belongs to the machine, not to any one film, and it is neither backed up nor synced anywhere.
<code>CIN_Projects_v1</code>	The index of every project on this machine. Also outside the prefix, and not one production's to carry.

The two `SB_` exclusions are enforced twice — in the vault's `LOCAL_ONLY` pattern and again in the cloud function — because either side alone is one bug away from publishing a key to every owner.

A backup is readable by anyone holding it. The file is plain JSON, not encrypted and not password-protected. It contains the crew directory as entered: names, rates, telephone numbers, email addresses and emergency contacts, alongside deals, budget actuals and everything else the production has recorded. Treat the file as you would a printed deal memo, and think before sending one over an ordinary channel.

Restoring

Restore is destructive. Writing an archive removes *every* portable `SB_*` key in this browser and then writes what arrived. It is not a merge and there is no undo inside the workspace. The Projects page stashes the current workspace into its own slot before importing — that stash, or a backup file, is the only way back. An archive carrying no production data is refused outright rather than obeyed, because obeying it would clear the workspace and write nothing.

An archive from another owner or machine is treated as untrusted: every string is neutralised before a single key is written, with a short list of prose fields (script, notes, synopsis) kept exactly as typed. That happens in full before storage is touched, so a file that fails the check changes nothing.

The restore reports rather than reassures. It states how many media files were written, how many were unreadable and left out, how many this browser refused to store, and how many the production references but nobody has the bytes for. It claims to be complete only when all four of those are empty.

The studio cloud

The cloud is one shared namespace for five owner accounts, behind the owner login. This is the design, not an oversight: **every owner can list, pull, overwrite and delete every production**, including ones they did not

create. There is no per-owner partition and no private area. Pushing under a name that already exists replaces the cloud copy of that film, and the catalog records who saved it last.

Two protections exist, and they are narrow. A push may declare the version it believes it is replacing; if the cloud has moved on, the save is refused with a conflict rather than burying the newer one. And the badge holds back auto-sync when the cloud copy belongs to another owner and this tab has never taken it over. Neither prevents a deliberate Save to cloud from overwriting a colleague's work.

Versions: a ring of eight

Each push moves the copy it replaces into a rotating ring of eight superseded copies. History lists what is on file, with times and the owner who saved each; restoring puts one back, and the copy that was live at that moment is kept too, so the undo is not itself a way to lose work. A delete also lands in the ring, with a record of who deleted it and when.

The ring holds the written record, not the photographs. Eight copies of every still would be a storage bill rather than a backup policy. Restoring an earlier version brings back the text of the production; it does not bring media back, and the response says so plainly rather than letting an operator infer otherwise.

Delete removes the media permanently. Deleting a production from the cloud sweeps its media parts from the store. The written record stays recoverable through the ring; the photographs do not. This is deliberate and recent: the parts were previously left behind, which meant a production its owner believed deleted could still have its location interiors and wardrobe continuity fetched by anyone who had seen the name. That was a leak that looked like a safety net, so the sweep was made real and the deletion record now states that the media went. Export a `.cinamate` backup before deleting anything you may want the pictures of.

The four-minute auto-sync

Every module page runs a background sync on a four-minute interval (`SYNC_MS`), with a first pass around twenty-five seconds after load and one more attempt when the tab is hidden. It hashes the portable `SB_*` stores, does nothing if they have not changed, checks the cloud catalog, and pushes the active production under this owner's name.

It carries the written record only — never media. The server treats that silence as "this push is not about photographs" and carries the previous media state forward untouched, so the cloud copy's pictures remain whatever the last deliberate Save to cloud uploaded.

Working on two machines, take this literally: whichever machine has a Cinamate tab open is pushing its own idea of the production every four minutes. Pull before you start on the second machine, and close or reload the first — otherwise the version check will start refusing saves, or a quiet machine will push a stale record over a day's work. An archive approaching the cloud's four-megabyte ceiling is skipped by auto-sync entirely and becomes manual territory.

Limits that will stop you

LIMIT	VALUE	APPLIES TO
Written record	4 MB	One production's <code>SB_*</code> half in the cloud. Media travels separately and does not count towards it.
One media record	900 KB	Roughly three times a full-size scout JPEG. The cloud carries stills and short takes, not source masters.
One part	1 MB	A single upload request, sized to be cheap to retry.
Parts per production	48	Which is where the 48 MB per-production ceiling comes from — it holds by construction.

Anything over any of these is refused whole, before a single byte goes on the wire, with a message naming the offending file and its size, stating that nothing was sent, and offering the `.cinamate` export instead — which has no size limit. A production is never uploaded down to whatever happened to fit: an archive that arrives without its photographs must not look complete.

Not established from source. The vault engine and the cloud function are exercised by `scripts/test_vault_blobs.mjs` against an in-memory media adapter, so the pinned behaviours above are the engine's. How a real browser behaves under storage pressure — quota refusals, an IndexedDB write aborted mid-restore — is reported by the page when it happens but cannot be characterised by reading, and no figure for it is offered here.

The Producer Suite I — The Budget Top Sheet

A top sheet is a promise about money made in public. This chapter describes exactly how Cinamate arrives at every figure on it — which numbers you type, which the sheet derives, and the order the derivations run in, because that order moves the grand total.

On the accuracy of this chapter. Everything below was established by reading the source files named in it. The platform is being worked on continuously; if a figure on your screen contradicts a sentence here, the running software is the authority and this page is out of date.

Three levels, one stored sheet

The Budget tab of the Producer Suite (`producer/index.html`, driven by `producer/budget-sheet.js`) is a three-level hierarchy: the *top sheet* on the left, a list of account categories; the *detail table* on the right, the line items inside whichever category is selected; and beneath both, the derived totals. Selecting a row on the left changes what the right-hand table edits.

There is exactly one sheet. It autosaves to browser storage under the key `SB_BudgetSheet_v1`, on a 300 ms debounce after any edit. There is no version history and no undo — a replaced sheet is gone unless you exported it first.

The chart of accounts

Account numbers here are not local to this page. They come from `js/lib-money-accounts.js`, the single chart every module that posts money shares: crew deal memos, the casting office, VFX bids and the Money Room cost report all commit against these same numbers, which is what lets an actual find its way back to the budget line that anticipated it.

A new sheet carries nineteen major accounts. Eleven are *labour* accounts, and that flag defines the base payroll fringes are computed on.

ACCT	CATEGORY	LABOUR	ACCT	CATEGORY	LABOUR
1000	Story & Rights		11000	Makeup & Hair	yes
2000	Producers Unit	yes	12000	Transportation	yes
3000	Direction	yes	13000	Locations	
4000	Cast	yes	14000	Media & Stock	
5000	Production Staff	yes	15000	Post-Production	
6000	Camera	yes	16000	Insurance & Legal	
7000	Sound	yes	17000	Publicity	
8000	Grip & Electric	yes	18000	General Expenses	
9000	Art Department	yes	20000	Payroll Fringes	
10000	Wardrobe	yes	19000	Contingency	computed

Two absences are deliberate. **19000 Contingency** is not a category you can type into — it is derived from everything else, so seeding explicitly discards any contingency line offered to it. **20000 Payroll Fringes** exists as its own account precisely because fringes are cross-departmental; folded into General Expenses they read as several times the whole of an account whose job is to be a small percentage of below-the-line.

The chart also carries *detail* accounts that departments post to, each rolling up to exactly one major above: 4100/4200/4400/4500 under Cast, 8500 under Grip & Electric, 9100 and 9900 under Art Department, 13500 under Locations, 15200/15400/15600/15800 under Post-Production, 16500/16800 under Insurance & Legal. Anything the chart does not recognise rolls up to itself, so it surfaces on the cost report as **Unbudgeted** rather than disappearing into a real line.

Line items — Amt × Units × Rate

Each line item stores a description, three calculator fields (Amt, Units, Rate), an Estimated figure, an Actual figure and free-text notes. The rule that decides what a line is worth lives once, in `js/lib-money-sheet.js`:

- If **all three** of Amt, Units and Rate are greater than zero, the line is worth $\text{Amt} \times \text{Units} \times \text{Rate}$, and the Estimated cell becomes a read-only computed display.
- Otherwise the line is worth whatever is typed into Estimated, which stays an editable field. This is the lump-sum path.

A zero or a negative in any of the three calculator fields therefore switches the line back to the manual figure. Numbers are read tolerantly — 1234.56, \$1,234.56 and 1 234.56 all parse the same, and accounting parentheses (50) read as minus fifty. Anything genuinely unreadable reads as zero; a budget cell is never allowed to become NaN.

On save, the computed figure is written back onto the line item's stored Estimated value. That is not cosmetic: five readers across finance, the investor letter and the learning layer sum the stored value alone, so before this a fully built sheet of calculator lines read as zero everywhere outside this page.

Category subtotals and the percentage column

A category subtotal is the sum of its line items' values, with an Actual subtotal alongside it summed the same way. The percentage at the right of each top-sheet row is that category's share of the *grand total* — not of the direct subtotal — rounded to a whole percent.

The derived layers, and the order they run in

Four figures are computed rather than typed. The order matters, and it is fixed in `sheetTotals()`:

1. **Direct subtotal** — every line item in every category.
2. **Payroll fringes** — the fringes percentage applied to the *labour base*, which is the subtotal of the eleven labour accounts only. Not the whole sheet.
3. **Completion bond** — the bond percentage applied to the direct subtotal.
4. **Insurance** — the insurance percentage applied to the direct subtotal.
5. **Contingency** — applied last, to the sum of the four figures above. Contingency therefore covers the fringes, the bond and the insurance, not merely the direct costs.

Bond and insurance quote on the direct subtotal and nothing else, so raising the fringes percentage does not move either of them. Worked through on a round million, with a labour base of \$400,000:

LAYER	BASE	RATE	AMOUNT
Direct subtotal	—	—	1,000,000.00
<i>labour base (a subset, not added again)</i>	—	—	<i>400,000.00</i>
Payroll fringes	labour base	28%	112,000.00
Completion bond	direct subtotal	2%	20,000.00
Insurance	direct subtotal	2.5%	25,000.00
Contingency basis	sum of the four	—	1,157,000.00
Contingency	basis	10%	115,700.00
Grand total			1,272,700.00

Had contingency been taken on the direct subtotal instead — the order most hand-built spreadsheets use — it would be \$100,000 and the grand total \$1,257,000. The same inputs, \$15,700 apart. When you quote a top sheet against someone else's, establish which order they used before arguing about the number.

All four percentages are edited in the header strip above the top sheet, and each is clamped: contingency 0–30%, fringes 0–45%, bond 0–6%, insurance 0–8%. A value outside the range is pulled back to the nearest end rather than rejected.

Fringes default to zero, on purpose. A new sheet's fringes, bond and insurance percentages are all zero, while the default skeleton already contains a *Cast fringes* line under 4000 and a *Payroll fringes (labor accounts)* line under 20000. Fill in those line items or set the percentage — doing both counts your fringes twice, and nothing warns you.

Why every figure is held in cents

All arithmetic on this sheet runs through `js/lib-money-math.js`, which carries money as an integer number of cents. Rounding happens once, half away from zero, at the moment a fraction of a cent is created — a multiplication or a percentage — and never on a sum. A total is then the sum of the cents that were actually posted, so it foots by construction rather than by luck.

This is not fastidiousness. A JavaScript number is a binary float: $3 \times 5 \times 1061.64$ is 15,924.6, and left as a float it prints as 15924.599999999999. Rounding each row before adding, rather than adding what was posted, produced a cost report whose TOTAL row disagreed with its own columns by tens of dollars across a couple of hundred accounts — on the report that goes to the completion bond.

Money therefore leaves the platform as a fixed two-decimal *string*, not as a raw number, and that is what makes an exported column safe to sum. The test suites assert it: `scripts/test_ops_money.mjs` requires every money cell in an exported report to match `-\d+\.\d{2}` with no cell anywhere carrying three or more decimals, and `scripts/test_producer_suite.mjs` requires the $3 \times 5 \times 1061.64$ line to export as 15924.60.

Actuals against estimates

Every line item carries an Actual column beside its estimate, and category and sheet actuals are summed from those. When the sheet holds any actual, the grand total row prints it alongside the estimate.

If the Money Room has posted anything — purchase orders, petty cash or payroll in `SB_Money_v1` — the top sheet also prints a one-line cost summary beneath the totals: estimated final cost, and the variance marked *over* or *under*, linking through to the finance page. That summary is computed by the Money Room's own cost report, not recomputed here.

Actuals also feed the estimator's learning layer (`js/learn.js`, stored under `CIN_Learn_v1`), which learns a per-account correction from lines that have both an estimate and a real actual. It is gated: a production teaches only once, when it has been marked wrapped in `CIN_Projects_v1`. Mid-shoot, a department that has spent 12% of its budget in week two looks like an 88% underrun, and folding that in would drag every future estimate down.

Cash needed by phase

Below the grand total the sheet prints a three-way split — prep, shoot, post — of when the money actually leaves the bank. Each account has a fixed phase profile (rights are entirely prep, post-production is almost entirely post, cast is overwhelmingly shoot), and the derived layers that sit above the categories are split between shoot and post, with the last bucket absorbing the remainder so the three figures add back to the grand total exactly.

The norms advisor

Each top-sheet row is measured against a band of typical shares of the grand total for that account, held in the `NORMS` table in `producer/budget-sheet.js`. A category outside its band gets a marker: ▲ for heavy, ▼ for light, with the band itself in the marker's tooltip, and a line beneath the totals counts how many accounts are flagged. Being outside a band is not an error. It is the question the bond company will ask, asked early.

Seeding from the script estimate

Seed from script estimate runs the script-driven estimator over the current project, routes every estimated line to the top-sheet category its account rolls up to, and writes the **midpoint** of each estimated low–high range into the line, with the range itself preserved in that line's notes. Where the learning layer has at least two past actuals on an account and a correction other than 1.0, the seeded figure is multiplied by it and the note records by how much. Contingency lines are dropped, since contingency is derived. If the stripboard on the Schedule tab has real day assignments, the estimator uses them, and the confirmation reports what the cast drops saved in weeks.

Seeding replaces the whole sheet. Every line item in every category is deleted before the estimated lines are written, and the contingency percentage is reset to 10%. Anything you had typed by hand is lost, including actuals. Export first — *Save (.json)* — if the sheet holds work you cannot retype.

The CSV export

Export CSV downloads the sheet as `<sheet name>.csv`, with the name stripped of anything outside letters, digits, hyphen and space. The columns are `Account`, `Category`, `Line item`, `Amt`, `Units`, `Rate`, `Estimated`, `Actual`, `Notes`. Each category writes one row per line item, followed by a `SUBTOTAL` row when it holds anything. Then, at the foot: `SUBTOTAL (all categories)`; a `fringes`, `bond` and `insurance` row each, but only when that percentage produced a figure; a `contingency` row always; and `GRAND TOTAL`. The `Actual` column is left empty rather than written as `0.00` when there is nothing in it.

Two things about that file are worth knowing before you open it.

The money columns are text-formatted numbers

Estimated and Actual are written as fixed two-decimal strings, for the reason given above. A spreadsheet reads them as numbers and sums them correctly. What you will not see is a float tail, and that is the point.

The leading apostrophe is deliberate

A cell whose first character is `=`, `+`, `-`, `@`, a tab or a carriage return is treated by Excel and Google Sheets as a *formula*, not as text. A line item description typed into this sheet would then execute on the machine of whoever opened the export. The exporter therefore prefixes such a cell with a single apostrophe, which both spreadsheets read as "the rest of this cell is text".

Do not strip those apostrophes. If you see a cell reading `'=Second unit day`, that is the neutralisation working as designed. One consequence to expect: a genuinely negative money figure exports as `' -1234.56`, because it too begins with a hyphen, and a spreadsheet will treat it as text rather than as a number. If you need a credit to behave arithmetically in Excel, re-enter that one cell after opening the file.

Saving, loading, printing, starting over

Save (.json) writes the complete sheet — categories, line items, percentages, actuals — to `<sheet name>.budget.json`. That file is the archive copy; the CSV is a report, and cannot be loaded back. *Print / PDF* hands the rendered sheet to the browser's print dialogue.

Load and New sheet both replace the live sheet. *Load* accepts a `.json` file and, if it parses and contains a categories array, swaps it in immediately — with no confirmation prompt. *New sheet* asks first ("Start a new blank top sheet? The current sheet will be replaced (save it first if needed).") and then discards everything. Both write straight to `SB_BudgetSheet_v1`. There is no undo.

What this sheet is not

Stated plainly, because a document full of account numbers invites the assumption:

- **It is not an accounting system.** It holds one estimate and one actual per line. It has no ledger, no double entry, no reconciliation and no audit trail. The Money Room tracks commitments and spend; neither is a set of books.
- **It files nothing.** Nothing here is submitted to a jurisdiction, a payroll company, a guild, a bond company or a tax authority. Every export is a file that lands in your downloads folder.
- **The incentive figures are estimates from a table.** The Incentives tab computes recovery as $\text{budget} \times \text{qualified-spend percentage} \times \text{credit rate}$ against a stored table of headline program terms, and flags a budget below a program's minimum spend or above its cap. Real programs carry qualification rules, allocation queues, sunset dates and audits that no table models. Treat the output as a shortlist to discuss with a production accountant, never as advice.
- **The on-screen figures are abbreviated.** The top sheet displays money in shortened form — a grand total of \$1,272,700 reads as `$1.3M`, a line of \$15,924.60 as `$15.9k` — and clamps a negative to zero for display. The exact figures live in the CSV and the JSON. Quote from the export, not from the screen.